Assignment #5. JavaScript Basics
(DOM Manipulation & Events)

Objective:

By the end of this assignment, students will be able to:

- use variables, operators, random numbers, conditions and strict equality in JavaScript;
- organize code into functions and process arrays of data with loops;
- create, update and remove page elements through the DOM;
- handle user events and validate a form in real time with clear, accessible feedback.

Each task has several options: choose one of them or one of the other ideas. If you want to do your own idea, agree on it with the instructor first. You can connect the tasks to the project you started for the Midterm and add them to it, or do the assignment separately, without the project. In a team project, add them only to the page you develop yourself.

Rules for Submission:

1. This is an individual assignment. If the works of different students are the same, the grade is 0.
2. Submit before the deadline. **Late works are not accepted!** To get a grade you must **submit before the deadline AND defend your work at the practice lesson**. Submission without a report or without a defense will result in a grade of 0.
3. You must submit: your report in **Assignment5_YourName.docx** (or .pdf) format. If you submit in other formats, your grade will be 0.
4. All source code must be uploaded to the GitHub repository. Put the JavaScript of each task in its own file.
5. If you cannot explain your own code, the whole assignment is graded 0.
6. IN THE REPORT MUST BE:
    - Your name and group, and the option you chose for each task
    - Every task with screenshots and a short explanation in your own words
    - Task 3: the table of your rules
    - A short conclusion about what you learned
    - The **GitHub repository URL** (and the GitHub Pages link if the tasks are in your project). Submissions without the link will receive a grade of 0.

Task 1. Browser game (variables, operators, conditions)

Make a small game on the theme of your website or on any topic. Choose one option.

Option	What to build
A. Higher or Lower	Guess whether the next random item (a price, a year, a rating) is higher or lower. 3 lives, a streak counter and the best streak; equal values count as a right guess.
B. Turn-based battle	Hero and enemy with 100 HP. The hero attacks (8–15), defends (halves the next hit) or heals (+20 HP, up to 3 times); the enemy hits back for 5–14. A 10% chance of a critical hit. HP bars and a log of the last 5 actions.
C. Code breaker	Guess a 4-digit code with no repeated digits in 8 attempts. After each guess, show how many digits are in the right place and how many are in the wrong place.

Other ideas: rock-paper-scissors with five moves, a quiz with a 50/50 hint, a board race, a word-guessing game, a memory card game.

Requirements:

- at least three outcomes decided by conditions with &&, || and ===;
- all numbers of the game stored as constants at the top of the script;
- fair random values, the smallest and the largest included;
- invalid input or moves rejected with a message, without changing the game state;
- the state shown after every move; the game stops at the end and restarts without reloading.

Hint: **Math.floor(Math.random() * (max - min + 1)) + min** returns a whole number from min to max.

Tools: an external JS file with defer, let and const, Math.random, if / else, addEventListener.
No jQuery, no libraries and no onclick attributes.

Task 2. A page built from data (arrays, loops, functions, DOM)

Store the content of your project page (or of any topic) in a JavaScript array of objects and build the page from it. Choose one option.

Option	What to build
A. Kanban board	Cards move between three columns, e.g. To do, In progress and Done. The middle column holds at most 3 cards. Sorting by priority, a filter by tag and a progress bar of finished cards.
B. Configurator	Build a set by choosing one item per category, e.g. a computer, a combo meal or a trip. Incompatible items are blocked with a reason. A live total within a budget set by the user; up to 3 builds saved and compared.
C. Seat map	A hall or a bus drawn from a 2D array with price zones. Up to 6 free seats selected, with a live total. A Best seats button finds free seats together, closest to the center.

Other ideas: a tournament table, a crafting or recipe system, a playlist with total duration, a trip planner with a budget, an achievements tracker.

Requirements:

- at least 10 objects with a unique id and at least four properties of different types; the HTML has only empty containers;
- one render function, called after every change;
- a search or a filter, with a message when nothing is found;
- at least one action blocked by a rule (a limit, a conflict, a budget), with the reason shown;
- at least one value calculated with your own loop;
- one listener on the container for the buttons of all items (event delegation).

Hint: change the array first and then call **render()**: the page always shows the current data.

Tools: arrays of objects and their methods, functions, for and for…of loops, createElement, textContent, classList, dataset. No innerHTML for the data.

Task 3. Form with live validation (events, form validation)

Build a form that shows the result while the user types, explains every mistake and creates the final result only from correct data. Choose one option.

Option	What to build
A. Ticket builder	Event, date, ticket type, quantity, name and email; a ticket preview updates while typing. Each type has seats left (an array of objects): the quantity cannot exceed them. Sold-out dates are rejected; a promo code from a list gives a discount. Result: a ticket with a random booking code.
B. Character creator	Name, class, color and four skills; a character card preview updates while typing. Exactly 20 skill points, each skill from 1 to 10, with a live counter of points left. Taken names (an array) are rejected; each class needs a minimum skill, e.g. a mage needs magic 5 or more. Result: the final character card.
C. Application wizard	Three steps with a progress bar: Next checks only the current step, Back keeps the data. One rule compares fields from different steps. A summary panel next to the form updates while typing. A review step with Edit links. Result: a confirmation with an application number.

Other ideas: a CV builder with a live preview, an appointment form with taken time slots, a team registration with unique emails, a delivery order with promo codes.

Requirements:

- at least six fields and four rules that HTML attributes cannot express;
- a live preview that changes on every input;
- no errors on page load; a field is checked on blur, then on every input, with a clear message under it and aria-invalid;
- on submit: with errors, their number at the top and the focus on the first one; on success, the result built with the DOM and a reset form;
- HTML attributes kept, browser messages turned off with **form.noValidate = true**; full keyboard support.

Report: a table of your rules with field, rule, valid and invalid examples.

Hint: form.noValidate = true turns off only the browser messages; **checkValidity()** still works.

Tools: submit, input, blur and change events, preventDefault(), the Constraint Validation API.
No jQuery and no libraries.

Assessment criteria

Criterion	What is checked	Points
Task 1	- the rules of the chosen option work, also after the constants are changed (6) - the game meets all the requirements of the task (6)	**12**
Task 2	- the rules of the chosen option work with the initial data and with new data (6) - the page meets all the requirements of the task (6)	**12**
Task 3	- every rule gives the right result for the student's table and for new values (6) - the form meets all the requirements of the task (6)	**12**
Styling and quality	- consistent custom styling: colors, fonts, spacing; on a project page it matches the website (3) - every state is styled: hover, focus, disabled, error, success, win/loss, empty list; JavaScript toggles classes, no inline styles (3) - works at 375px and 1280px without horizontal scroll; no console errors and no bugs after repeated clicks and restarts (3)	**9**
Report	- contains everything listed in the rules; screenshots are readable; explanations are in the student's own words	**5**
Defense	- explains the work and demonstrates it live without errors (10) - answers the instructor's questions about their own code and the topic (15) - completes the live coding task independently, without assistance or hints (25)	**50**
Total		**100**

Resources:

1. Abitova G.A. Web technologies Front-End Development. Part 2 (2022).
2. Loiane G. Learning JavaScript Data Structures and Algorithms. 3rd ed. (2018).
3. https://learn.astanait.edu.kz
1. https://www.w3schools.com/js/js_random.asp
2. https://www.w3schools.com/js/js_if_else.asp
3. https://www.w3schools.com/js/js_arrays.asp
4. https://www.w3schools.com/js/js_htmldom.asp
5. https://www.w3schools.com/js/js_htmldom_eventlistener.asp
6. https://www.w3schools.com/js/js_validation_api.asp
7. https://getbootstrap.com/docs/5.3/forms/validation/

Good luck!